

Geospatial Weather and Elevation Analysis

Interim Progress Report — Week 4

Project optionOption 5

Team membersVidura · Kaumindi · Thishakya

Submission date16 July 2026

Reporting periodWeeks 1–4

ImplementationPython · Scala · Apache Spark

Milestone status ON TRACK

Reproducible setup, ingestion pipeline, Spark transformations, automated tests, and documentation are implemented.

0 Contents

1	Executive Summary	3
2	Project Scope and Objectives	4
3	Work Completed Through Week 4	5
3.1	Week 1 — Reproducible Setup	5
3.2	Weeks 2–3 — Data Ingestion	5
3.3	Weeks 3–4 — Spark Development	5
4	System Design	7
4.1	Core Data Schema	7
5	Verification and Current Evidence	8
6	Risks, Limitations, and Mitigations	9
7	Plan for Weeks 5–6	10
8	Reproducibility	11
9	Data Attribution	12

1 Executive Summary

This report records progress on the Geospatial Weather and Elevation Analysis project through the end of Week 4. The objective is to quantify relationships between terrain elevation and temperature, precipitation, and wind across geographically diverse locations. The system uses Open-Meteo as its data source, Python for API ingestion, Parquet for storage, and Scala with Apache Spark for distributed processing and statistical analysis.

The Week 4 milestone has produced a runnable project rather than a design-only prototype. The repository now contains a curated 36-location study configuration, cached and retry-safe API clients, year-partitioned Parquet ingestion, explicit validation rules, elevation and seasonal enrichment, grouped summaries, 30-day rolling metrics, annual year-over-year changes, and seasonal linear regression for environmental lapse-rate estimation. Unit tests cover the ingestion schema, partition layout, elevation-band boundaries, hemisphere-aware seasons, and rejection of invalid observations.

The complete historical extraction produced 1,893,888 hourly records across all 216 location-year partitions. Spark processed the full dataset successfully, while a short, repeatable sample command remains available for rapid pipeline verification. Dashboard, PostgreSQL integration, and end-to-end usability testing remain scheduled for Weeks 5–6.

2 Project Scope and Objectives

The project investigates how altitude and location influence weather. Four elevation bands are used consistently throughout processing:

Band	Range	Analytical purpose
Lowland	0–199 m	Coastal and near-sea-level baseline
Midland	200–999 m	Moderate-altitude transition
Highland	1,000–2,999 m	Strong terrain influence
Alpine	$\geq 3,000$ m	Extreme high-altitude conditions

The primary questions are:

- How rapidly does temperature change with elevation, and how does that lapse rate vary by season?
- Do precipitation totals show consistent gradients between elevation bands and regions?
- Is mean wind speed amplified at higher elevations?
- How do annual and rolling temperature patterns differ between locations?

The study period remains 1 January 2019 to 31 December 2024. This is six complete calendar years; the proposal described it as five years, so the implementation and this report correct that counting error while retaining the proposed dates.

3 Work Completed Through Week 4

3.1 Week 1 — Reproducible Setup

COMPLETE A conventional project structure and pinned dependency ranges have been added. Python installation uses a local virtual environment and an installable package. The Spark application uses sbt with Scala 2.13 and Apache Spark 4.0.1. Common commands are exposed through a Makefile, and generated data, caches, secrets, and build outputs are excluded from version control. The project documentation defines prerequisites, commands, storage layout, units, attribution, and the boundary between the Week 4 milestone and later dashboard work.

3.2 Weeks 2–3 — Data Ingestion

COMPLETE The ingestion layer implements the three historical-data endpoints described in the proposal:

- The Geocoding API resolves each configured place and filters results by country code.
- The Elevation API retrieves Copernicus DEM GLO-90 terrain height for the resolved coordinates.
- The Historical Weather API retrieves hourly temperature at 2 m, precipitation, wind speed at 10 m, and relative humidity at 2 m.

Requests are cached locally, throttled after uncached responses, and retried up to five times with exponential back-off for transient failures. Historical calls explicitly select ERA5 so changes between model families do not introduce artificial time-series discontinuities and all four proposed weather variables remain available. Output is standardised to UTC, degrees Celsius, millimetres, metres per second, and percent.

Thirty-six target locations are configured across South Asia, East and Southeast Asia, Europe, Africa, North and South America, and Oceania. A live resolution audit confirmed all 36 locations and an elevation range of 4–4,410 m: 11 lowland, 4 midland, 9 highland, and 12 alpine sites. Locations are written once as metadata; hourly observations are written in a partition layout of `location_id=<id>/year=<year>`. This creates 216 independently recoverable partitions for the full study (36 locations × 6 years).

3.3 Weeks 3–4 — Spark Development

COMPLETE The Scala/Spark job now performs the transformations committed to in the proposal:

1. Reads self-describing Parquet data and checks the required schema.
2. Rejects null keys, duplicate location–time records, and physically impossible values.
3. Broadcast-joins location and elevation metadata on `location_id`.
4. Assigns the four elevation bands using explicit boundary rules.
5. Assigns meteorological seasons with the calendar reversed in the Southern Hemisphere.
6. Aggregates temperature, precipitation, wind, humidity, and observation count by elevation band, region, season, and year.
7. Builds daily metrics and a calendar-day 30-day rolling mean temperature per location.
8. Computes annual average temperature and year-over-year change per location.
9. Fits a Spark ML `LinearRegression` model of daily temperature against elevation for each season, exporting slope, lapse rate per kilometre, intercept, R^2 , RMSE, and sample size.

10. Writes enriched hourly observations partitioned by elevation band and stores each summary product as Parquet.

4 System Design

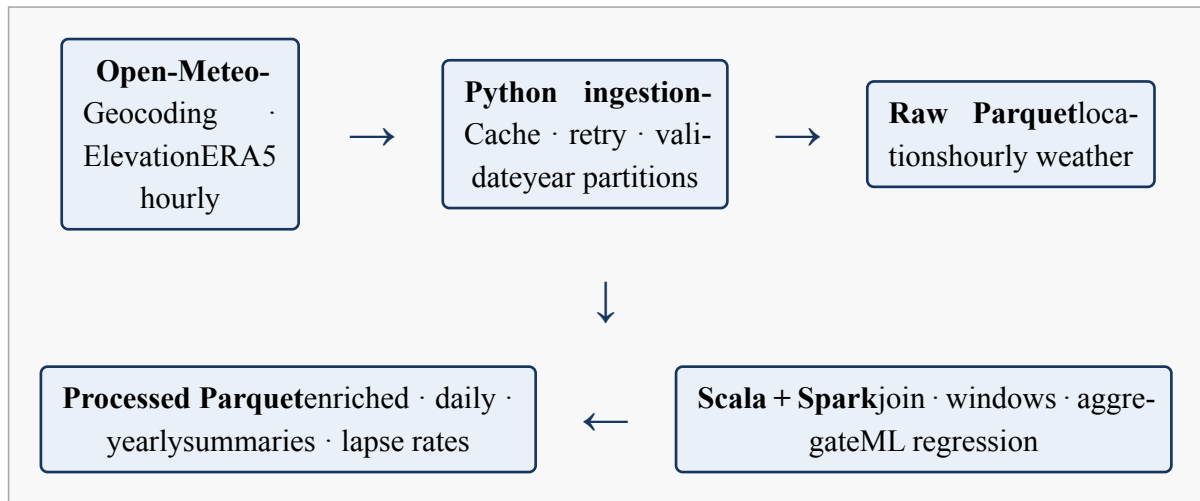


Figure 1: Implemented data flow through the Week 4 milestone.

The separation between raw and processed zones preserves source observations while allowing transformations to be rerun. Location and year partitions make failures recoverable without repeating the full extraction. Parquet is used throughout because it preserves types, supports column pruning, and is read and written natively by Spark.

4.1 Core Data Schema

Field	Type	Meaning
location_id	string	Stable join and partition key
timestamp	timestamp	Hourly observation time in UTC
temperature_2m	double	Air temperature at 2 m (°C)
precipitation	double	Preceding-hour total (mm)
wind_speed_10m	double	Instantaneous wind speed (m/s)
relative_humidity_2m	double	Relative humidity (%)
source_year	integer	Recoverable source partition
ingested_at	timestamp	Data-lineage timestamp

5 Verification and Current Evidence

Verification is automated at both language layers. Python tests construct controlled API-shaped input and confirm the stable column order, timestamp typing, partition path, row counts, and failure on missing variables. Scala tests start Spark in local mode and verify all elevation thresholds, Northern/Southern Hemisphere seasonal reversal, duplicate removal, and invalid-temperature rejection.

Verification item	Status	Acceptance criterion
Python unit tests	PASS	All ingestion tests complete without failure
Python static checks	PASS	No lint violations in ingestion source/tests
Scala/Spark unit tests	PASS	Transformation tests complete in local Spark
Sample API ingestion	PASS	Two locations × seven days written to Parquet
Sample end-to-end Spark run	PASS	All five processed datasets are produced
Full historical extraction	PASS	1,893,888 hourly rows across all 216 partitions

The complete Spark run produced 1,893,888 enriched hourly rows, 78,912 location-day rows, 216 location-year rows, 432 grouped summaries, and four seasonal regression records. Preliminary univariate lapse-rate estimates are shown below.

Season	Lapse rate (°C/km)	R ²	Daily records
Autumn	−3.088	0.426	19,752
Spring	−2.739	0.327	19,776
Summer	−3.506	0.717	19,712
Winter	−2.818	0.229	19,672

All fitted slopes are negative, consistent with temperature decreasing as elevation increases; the strongest relationship in this first model occurs in summer. These coefficients are preliminary rather than final scientific conclusions. A global one-variable model confounds elevation with latitude, region, and site selection, while the lower winter R² indicates substantial unexplained variation. Regional stratification and residual checks are therefore required before interpretation.

6 Risks, Limitations, and Mitigations

Risk	Potential effect	Mitigation
Modelled weather	Reanalysis is not a station measurement and smooths local extremes.	Use one consistent model, describe resolution, and avoid causal claims.
Terrain mismatch	A city coordinate may not represent a nearby summit or valley.	Retain resolved coordinates/elevation and inspect outliers before modelling.
Regional confounding	Latitude and climate region also affect temperature.	Report regional/seasonal strata; discuss multivariate extension.
API availability	Long extraction could be interrupted or rate-limited.	Persistent cache, annual partitions, throttle, retries, and resumable overwrites.
Unequal spatial coverage	More South American alpine locations may bias global summaries.	Show per-band sample sizes and interpret bands with regional context.

7 Plan for Weeks 5–6

Week	Planned work	Deliverable
5	Perform detailed quality profiling; load dashboard summaries into PostgreSQL; implement map, scatter, and time-series views.	Queryable database and Streamlit dashboard
6	Add current forecast search; integrate components; test correctness, performance, and usability; finalise analysis and documentation.	Tested application, final report, and presentation

The immediate next action is detailed distribution checks for missing values, suspicious elevations, and per-band representation, followed by regional regression and residual diagnostics. Scientific conclusions will be finalised only after those checks pass.

8 Reproducibility

From a machine with Python 3.11+, Java 17/21, sbt, and Typst installed, the milestone is reproduced with:

```
make install
make test-python
make ingest-sample
sbt test
sbt 'runMain lk.ac.ds4004.weather.WeatherJob'
make interim
```

The full proposed data range is collected separately with `make ingest-all`. API responses and generated datasets remain outside version control, while configuration, code, tests, and build definitions are versioned.

9 Data Attribution

Weather and geocoding data are provided by Open-Meteo. Historical analysis uses ERA5 reanalysis. Elevation is supplied through Open-Meteo from the Copernicus DEM 2021 GLO-90 dataset. The final dashboard and report will retain these attributions and add the required upstream citations.